

Software Development – Pattern-Based Questions

4 practice questions covering common software-development design patterns

Pattern 1 – Singleton Pattern

A software development team is building a **configuration manager** for an application. The application may create many components, but every component must access the same configuration instance. If multiple configuration objects are created, inconsistent settings may occur.

Question: Design a solution using the **Singleton Pattern**. Explain why Singleton is appropriate, how you would prevent multiple instances, and mention one advantage and one limitation.

Answer space / notes:

Pattern 2 – Factory Method Pattern

A notification system must support different notification types such as **Email, SMS, and Push**. The main application should not directly depend on the concrete notification classes. New notification types should be added with minimal changes to existing code.

Question: Apply the **Factory Method Pattern** to this problem. Identify the product, concrete products, creator, and concrete creators. Briefly describe how a new notification type can be added.

Answer space / notes:

Pattern 3 – Observer Pattern

An online stock-trading application maintains a live stock price. Several users and dashboard components need to be informed whenever the price changes. The number of users can change dynamically at runtime.

Question: Design this system using the **Observer Pattern**. Identify the Subject and Observers, describe the notification flow, and explain why Observer is better than manually updating every dashboard.

Answer space / notes:

Pattern 4 – Strategy Pattern

An e-commerce application calculates shipping charges using different strategies: **Standard**, **Express**, and **Same-Day**. The customer may change the shipping method before placing an order. The checkout code should not contain a large if-else chain for every shipping algorithm.

Question: Use the **Strategy Pattern** to design the solution. Identify the Context and Strategy, show how the shipping algorithm can be changed at runtime, and state two benefits of using this pattern.

Answer space / notes: